# Tell Me about yourself: SLOW and PAUSES

1- Back in France i got Master's degree SMS => companies get ISO-certified, I relocated to Canada.
2- Versatile employee => CSR, Tech support, Ticket + QA supporting testing..
**3- Love it so much that went back to school part time in 2017.**
4- Soon after i became **consultant** which open doors to various projects Ex:

Leading **Procedure Acceptance** testing incollab with buiz for TD management

Build Web **A**pp, software app and Design systems with AWS, Salesforces

Learn **Pr**ograming create test scrpts

*MABL, Blog Post, Test Automation University*

After reading carefully the job description, i am confident that

- It’s Amazing growth opportunity
- I have Skills and experience to meet requirements

# Day 1 - Goal

## 1. Understand Project Goal

Your company runs an e-commerce site. A critical user journey is the shopping cart and checkout process

- Option 1 => https://www.nopcommerce.com => cloudflare is restricting automation by adding captcha
- Option 2 => https://demo.simplcommerce.com/

## 2. Understand the test objective

- Validate core ecommerce flows, Ensure correct pricing and cart totals, Verify form validations and error handling, Ensure UI consistency
- "shift left" and become more autonomous
- DOR: Application accessible, instructions and deadline received - DOD: All critical test cases executed
- As small stories as possible

# Day 1 - Goal

## 3. Prepare the Test Design Document

- Create the test cases and steps for adding/removing items from the cart and completing checkout. - See Excel File Attached
  - Navigation (NAV), Authentication(AUTH), Product Details (PROD), Cart details (CART) Checkout Fow (CHECK)
- Use the test pyramid to Prioritize which test
  - Unit level: shopping cart logic that calculates the total price including tax and discounts, address form, email input
  - Component level: ex: Add to Cart button to ensure that when clicked, it correctly triggers a state change
  - Integration : Clicks "Place Order," successfully passed from the SimplCommerce application to the Stripe API
  - E2E: Manual functional testing , Cross-browser testing, Mobile simulator.
  - Out of scope: API, Advanced security testing

# Day 1 - Goal

| TEST AUTOMATION PRIORITY | | | | | |
|---|---|---|---|---|---|
| **Priority** | **Scenario** | **Business value** | **Risk** | **Frequency** | **Rational** |
| **1** | Checkout flow (TC_CHECK) | Direct revenue impact (orders, payments, coupons). | Very high — failures cause lost sales, financial issues, customer leaves. | Executed on every purchase; critical for regression. | Most complex flows (payments, addresses, coupons) + highest ROI for automation. |
| **2** | Cart details (TC_CART) | Core conversion funnel (add/update/remove cart). | High — pricing, quantity, or cart errors block checkout. | Very high — used by almost every user session. | Strong dependency for checkout tests and frequent UI regressions. |
| **3** | Authentications (TC_AUTH) | Enables personalization, checkout, wishlist, order history. | Medium–High — login or registration failures block users entirely. | High — login, logout, forgot password are common. | Stable flows, good candidates for reliable automation. |
| **4** | Product details (TC_PROD) | Drives product discovery and engagement. | Medium — sorting/filtering/search issues hurt UX but not immediate revenue. | High — browsing and search are common. | Many UI assertions; less critical than cart/checkout |
| **5** | Navigations (TC_NAV) | Informational and routing only. | Low — issues are visible and easy to catch manually. | Medium. | Simple, low ROI for early automation; good for smoke tests later. |

| Test Scenario ID | TC_CHECK | | | | Created By | | | | |
|---|---|---|---|---|---|---|---|---|---|
| Description | Checkout flow | | | | Created Date | | | | |
| Date Tested | | | | | Version | | | | |

| Testcase ID | Summary | Pre-Requisite | User Input | Steps | Expected Result | Actual Result | Type | Status | Comments |
|---|---|---|---|---|---|---|---|---|---|
| **TC_CHECK_001** | Apply valid coupon | Product added to cart | Valid coupon code | • Navigate to Cart page<br>• Enter valid coupon code<br>• Click **Apply** | • Coupon is applied successfully<br>• Discount is reflected in order total | | | | |
| **TC_CHECK_002** | Apply invaild coupon | Product added to cart | Valid coupon code | • Navigate to Cart page<br>• Enter valid coupon code<br>• Click **Apply** | • Coupon is applied successfully<br>• Discount is reflected in order total | | | | |
| **TC_CHECK_003** | Order placement | User logged inProduct added to cart | Valid shipping, billing & payment details | • Navigate to Cart page<br>• Click **Checkout**<br>• Enter shipping & billing details<br>• Select payment method<br>• Click **Place Order / Pay** | • Order is placed successfully<br>• Order success / confirmation page is displayed | | | | |
| **TC_CHECK_004** | Proceed checkout without login | Product added to cart | N/A | • Add product to cart<br>• Click **Checkout** without logging in | • User is redirected to Login page<br>• Checkout is blocked until login | | | | |
| **TC_CHECK_005** | Add Shipping address | User logged inOn checkout page | Valid shipping address | • Navigate to Checkout page<br>• Enter valid shipping address details | • Shipping address is saved successfully<br>• Payment button becomes enabled | | | | |
| **TC_CHECK_006** | Add new billing address | User logged inOn checkout page | Valid billing address | • Select **Add new billing address**<br>• Enter valid billing details | • Billing address is saved successfully<br>• Billing address is applied to order | | | | |
| **TC_CHECK_007** | Pay with Card | User logged inOn checkout page | Valid card details | • Select **Card** as payment option<br>• Enter valid card number, expiry & CVV<br>• Click **Pay** | • Payment is processed successfully<br>• User is navigated to order success page | | | | |
| **TC_CHECK_008** | Cash on delivery | User logged inCOD available | Select COD | • Select **Cash on Delivery** payment option<br>• Click **Place Order** | • Order is placed successfully<br>• Success message displayed: **"We received your order. Thank you!"** | | | | |
| **TC_CHECK_009** | Pay with Paypal express | User logged inPaypal enabled | Paypal account | • Select **PayPal Express** option<br>• Login to PayPal<br>• Confirm payment | • Payment is completed successfully<br>• User redirected to order confirmation page | | | | |
| **TC_CHECK_010** | Checkout with empty Cart status | No items should be added in cart | | • Navigate to Checkout page | It should show message - "There are no items in this cart. Go to shopping" | | | | |
| | | Product | | • Navigate to Checkout page | | | | | |

# Day 2

1. Set up automation framework (Playwright with JavaScript/Typescript)
2. Design core cart scenarios:

- TC_CART_001: Adding a product to the cart.
- TC_CART_002: Removing a product from the cart.
- TC_CART_003: Updating product quantity (Data-Driven).
- TC_CHECK_001: Proceeding to checkout with valid details.
- TC_CHECK_010: Attempting checkout with missing/invalid details (no address / invalid payment).

# Day 2

## 1. Data-Driven Testing (Table-Driven Tests) with a simple for loop

PRO: Easy to scale (just add another object)

Con: Performance/ debug

Alternative: Import Faker to generate massive data and call it when needed

## 2. Scripted

PRO: Easy for demo, and for unstable domain

CON: Work as a Team

Alternative: Declarative for better reusability and maintanability

Day 2

## 3. Arrange–Act–Assert

PRO: Faster to write and simpler to address the platfrom stability.

Con: Reasoning lost,

Alternative: Given-When-Then (BDD) for better documentation

## 4. No Page Object Model

PRO: Huge files in the test

CON: Work as a Team

Alternative: POM to follow Playwright Recommendation

Day 2

## 5. Environment-Based Config

PRO: Demo

Con: Security

Alternative: Test Data Builders / Factories

# Day 3

1. Execute test suite and captures
2. Debugging
3. integrate into CI/CD
    - CI/CD Integration via YAML
    - Create Repo using GitHub
4. Create the documentation and refactor
6. Final review and clean-up

# Day 4

```
├── .github/workflows/       # CI/CD Pipeline configuration
├── Coding Testwares/        # Test design docs and screenshots
│   ├── Simplcommerce.xlsx   # Test case design & prioritization
│   ├── CICD pipeline.jpg    # CI/CD execution screenshot
│   ├── Report - logs and screenshot.jpg
│   └── Test Run.jpg         # Sample test execution
├── tests/                   # Test scripts (.spec.js)
├── playwright.config.js     # Global Playwright settings
├── .env.example             # Template for environment variables
└── README.md                # Project documentation
```

# Next Steps

- Flaky tests
- Run test using Docker to me more efficient
- Learn MCP protocol to enable agent
- Get stronger with Java and C# based => "polyglot" vs single-tool specialist